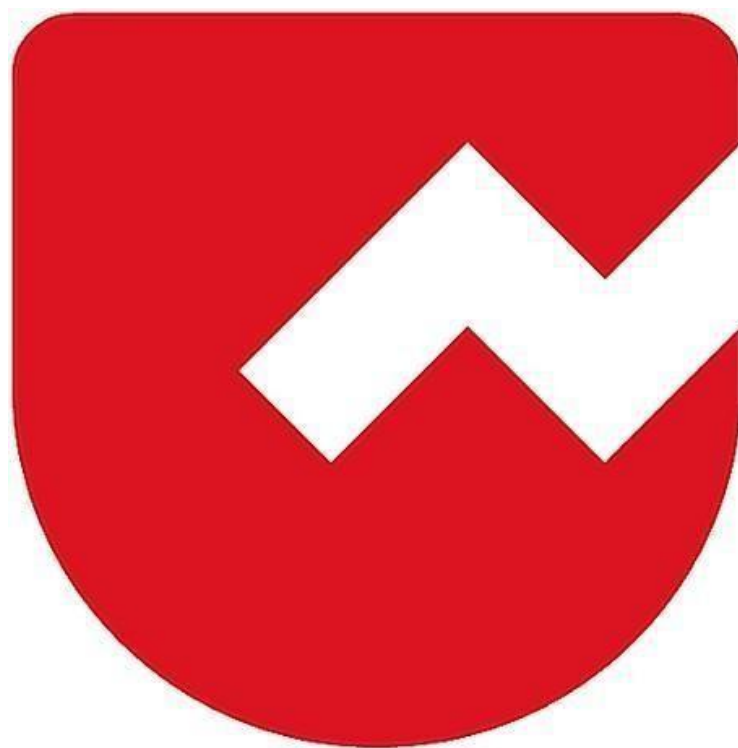




Zoho Schools for Graduate Studies



Notes

Date:23/9/2025
Day:Tuesday

Method Overriding

Overridden method :

The **super class** method that is subjected for **redefinition** is termed as overridden method.

Overridden method :

The **inherited sub-class** method that got **redefined** is termed as overridden method.

When to override a method ?

To capture the **unique child behaviour** that is different from the **parent**, we need to go for overriding.

Exceptions:

In Java, an exception is an event that occurs during the execution of a program that **disrupts the normal flow of instructions**. These exceptions can occur for various reasons, such as invalid user input, file not found, or division by zero. When an exception occurs, it is typically represented by an object of a subclass of the **java.lang.Exception class**.

There are two types of exceptions in java ,

- **Checked Exception**
- **Unchecked Exception**

Checked Exception :

Those exceptions that compiler must check for a **exception handler**, Except **Run-time exception** are all checked Exceptions and those exceptions must handled by the programmer with the Try and catch block , here **catch** block is a **exception handler**.

Some common checked exceptions are,

- **IOException** → File handling errors.
- **SQLException** → Database access errors.
- **ClassNotFoundException** → Class not found when loading dynamically.
- **InterruptedException** → Thread interruption.

Unchecked Exception :

Unchecked exceptions are exceptions that occur at **Run-time** and are not checked by the compiler during **compile-time**, Runtime exception and its subclass are all belongs to unchecked exceptions.

Some common unchecked exceptions are,

- **ArithmeticException** → Divide by zero.
- **NullPointerException** → Using a null object.
- **ArrayIndexOutOfBoundsException** → Invalid array index.
- **NumberFormatException** → Converting invalid string to number.
- **IllegalArgumentException** → Passing invalid argument to method.

Legal rules for Method Overriding :

1. Method overriding is possible if and only if the super class method is inheritable.
2. Overridden methods must share the same name and parameter list.
3. Overridden methods can have the same access level or a broader access level
4. Overridden methods can have the same return type or any sub-type of the super class return type (co-variant return types).
5. Methods that are marked as final or private or static cannot be overridden.
6. Constructors can be overloaded but cannot be overridden.
7. Overridden methods can throw any new unchecked Exception.
8. Overridden methods can eliminate throwing any unchecked Exceptions.
9. Overridden methods can throw any number of unchecked Exceptions.
10. Overridden methods cannot throw any new checked Exceptions.

11. Overridden methods can throw any sub-type of the super class checked Exception type.

12. Overridden methods can eliminate throwing any checked Exceptions.

Examples :

- **Rule 7 : Overridden methods can throw any new unchecked Exception.**

```
public class Animal // Base class or Parent class or Super class
{
    public Number eat() throws NullPointerException // overridden method
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() throws NullPointerException , ArithmeticException
    { // method overriding
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

An overridden method in a **subclass is permitted to throw any new unchecked exceptions, even if the superclass method does not.** This is because unchecked exceptions are not required to be handled at compile time, providing flexibility in how subclasses handle runtime errors.

- **Rule 8 : Overridden methods can eliminate throwing any unchecked Exceptions.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat() throws NullPointerException , ArrayIndexOutOfBoundsException
    {
        System.out.println("Animal eats");
        return 23;
    }
}
```

```
public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() // method overriding
    {
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

The Dog class is overriding the eat() method from its parent class, Animal. The overridden method in Animal declares that it can **throw two unchecked exceptions**: NullPointerException and ArrayIndexOutOfBoundsException. However, the overridden method in **Dog does not declare any exceptions at all**. This is a perfectly legal and valid action in Java due to the rules of method overriding.

- **Rule 9 : Overridden methods can throw any number of unchecked Exceptions.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat()
    {
        System.out.println("Animal eats");
        return 23;
    }
}
```

```

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() throws NullPointerException,
        ArrayIndexOutOfBoundsException // method overriding
    {
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}

```

Explanation :

unchecked exceptions are not part of the method's compile-time contract, **an overridden method can add as many of them as needed**. The Dog class's eat() method is within its rights to declare that it can throw these exceptions, **even though the parent method doesn't**. This doesn't violate the inheritance contract because clients of the Animal class are not required to handle these runtime exceptions in the first place.

- **Rule 10 : Overridden methods cannot throw any new checked Exceptions.**

```

public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat()
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override //overridden method
    public Byte eat() throws java.io.FileNotFoundException
    { // method overriding
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}

```

Explanation :

The Dog class is overriding the eat() method from its parent, Animal. The overridden method in **Animal doesn't throw any exceptions**. However, **the overridden method in Dog attempts to throw a new checked exception**, FileNotFoundException. This will result in a **compile-time error**.

- **Rule 11 : Overridden methods can throw any sub-type of the super class checked Exception type.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat() throws java.io.IOException
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derived class or child class or sub class
{
    @Override // overridden method
    public Byte eat()throws java.io.FileNotFoundException
    { // method overriding
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

The Dog class is overriding the eat() method from its parent, Animal. The overridden method in Animal throws a **checked exception**, IOException. The overridden method in Dog throws a **sub-type** of that exception, FileNotFoundException. This is a legal and valid action in Java.

- **Rule 12 : Overridden methods can eliminate throwing any checked Exceptions.**

```
public class Animal // Base class or Parent class or Super class
{
    // overridden method
    public Number eat() throws java.io.IOException, java.io.EOFException
    {
        System.out.println("Animal eats");
        return 23;
    }
}

public class Dog extends Animal //Derrived class or child class or sub class
{
    @Override // overridden method
    public Byte eat() // method overriding
    {
        System.out.println("Dog eats meat, milk, biscuit...");
        return 34;
    }
}
```

Explanation :

The Dog class is overriding the eat() method from Animal. The overridden method in the **parent class Animal throws the checked exception IOException**. However, the overridden method in the **child class Dog does not throw any exceptions at all**.

Thank You 😊